

Prácticas de Ampliación de Sistemas Operativos • Validación

La utilidad `asosystest`

La utilidad `asosystest` permite probar un programa, por ejemplo, `tarea.c`, de manera sistemática y reproducible. Para hacerlo, se tiene que proporcionar un archivo de configuración como entrada con el conjunto de pruebas en formato JSON. Por defecto, `asosystest` ejecuta todas las pruebas descritas en el archivo de configuración. Sin embargo, se pueden ejecutar una o más pruebas arbitrarias con la opción `-t` seguida de una lista de números de test o rangos de números (p.e. 2-5) separados por comas. Un ejemplo podría ser `-t 1,3-5,7` para ejecutar los tests 1, 3, 4, 5 y 7. La opción `-d` proporciona información detallada sobre la ejecución de los tests.

```
$ ./asosystest.py -h
asosystest.py: Version: v0.23.1
usage: asosystest.py [-h] [options]

asosystest testing module v0.23.1.

options:
  -h, --help                show this help message and exit
  -i TEST_FILE, --in-test-file TEST_FILE
                           JSON file containing list of tests.
  -t TESTIDS, --testids TESTIDS
                           List of ranges of test IDs.
  -d, --debug               Enable debug mode.
  -s, --setup               Execute setup commands only.

Example: asosystest.py -i example.json -t 1,3-5,7
```

El formato del archivo de configuración es el siguiente:

```
{
  "setup": {
    "desc": "Archivo de configuración para validación de 'tarea'",
    "timeout": 1,
    "cmds": ["touch f1"],
    "binaries": ["tarea"]
  },
  "tests": [
    {
      "cmt": "Test existencia f1",
      "cmd": "ls f1",
      "out": [
        "f1\n"
      ],
      "rc": 0
    },
    {
      "cmt": "Test salida 'tarea'",
      "cmd": "./tarea # Argumentos",
      "out": [
        "Hello, ASO!\n"
      ],
      "rc": 0
    }
  ]
}
```

La sección `setup` contiene la configuración para todas las pruebas:

- `timeout`: Límite de tiempo en segundos para cada prueba. Si la prueba no ha terminado pasado este tiempo, se interrumpe considerándose fallida.
- `cmds`: Comandos a ejecutar antes del inicio de las pruebas.
- `binaries`: Binarios necesarios para llevar a cabo las pruebas.

La sección `tests` es una lista de pruebas con los siguientes campos:

- `cmt`: Comentario(s).
- `cmd`: Línea de órdenes objeto de la prueba. Si se especifican varias líneas, éstas se ejecutan secuencialmente.
- `out`: Salida esperada en `stdout` y/o `stderr`.
- `rc`: Código de retorno esperado.
- `timeout` (opcional): Límite de tiempo en segundos para esta prueba.

Para ejecutar las pruebas de la sección `tests`, la utilidad `asosystest` crea un directorio temporal durante el procesamiento de la sección `setup`. De este modo, cualquier archivo creado durante la ejecución de las pruebas, se elimina cuando finaliza `asosystest`.

Validación de `tarea.c` con `asosystest`

Para ejecutar todas las pruebas del archivo de configuración `tarea.json`:

```
$ ./asosystest.py -i tarea.json
asosystest.py: Version: v0.23.1
asosystest.py: Created temporary directory: '/tmp/tmpba4mtli9'.
asosystest.py: Successfully copied binaries: '['tarea']'.
asosystest.py: Successful os.chdir('/tmp/tmpba4mtli9').
asosystest.py: Successfully executed setup commands '['touch f1']'.
asosystest.py: Archivo de configuración para validación de 'tarea'.T01:
Result    : OK!
asosystest.py: Archivo de configuración para validación de 'tarea'.T02:
Result    : OK!
asosystest.py: Successful tests: [1, 2]
```

Si modificamos `tarea.c` para que imprima `Hello, ASO!!!`:

```
$ ./asosystest.py -i tarea.json
asosystest.py: Version: v0.23.1
asosystest.py: Created temporary directory: '/tmp/tmpa469mivk'.
asosystest.py: Successfully copied binaries: '['tarea']'.
asosystest.py: Successful os.chdir('/tmp/tmpa469mivk').
asosystest.py: Successfully executed setup commands '['touch f1']'.
asosystest.py: Archivo de configuración para validación de 'tarea'.T01:
Result    : OK!
asosystest.py: Archivo de configuración para validación de 'tarea'.T02:
Result    : ERROR!
asosystest.py: Successful tests: [1]
asosystest.py: Failed tests: [2]
```

Para obtener más información acerca del error:

```
$ ./asosystest.py -i tarea.json -t 2 -d
asosystest.py: Version: v0.23.1
asosystest.py: Created temporary directory: '/tmp/tmpvxnn7env'.
asosystest.py: Successfully copied binaries: '['tarea']'.
asosystest.py: Successful os.chdir('/tmp/tmpvxnn7env').
asosystest.py: Successfully executed setup commands '['touch fl']'.

asosystest.py: Archivo de configuración para validación de 'tarea'.T02:
Result      : ERROR!
asosystest.py: Archivo de configuración para validación de 'tarea'.T02:
Command     : './tarea # Argumentos'
asosystest.py: Archivo de configuración para validación de 'tarea'.T02:
asosystest.py: Archivo de configuración para validación de 'tarea'.T02:
Expected (rc 0): ['H', 'e', 'l', 'l', 'o', ',', ' ', 'A', 'S', 'O', '!', '->', '\n']
asosystest.py: Archivo de configuración para validación de 'tarea'.T02:
asosystest.py: Archivo de configuración para validación de 'tarea'.T02:
Produced (rc 0): ['H', 'e', 'l', 'l', 'o', ',', ' ', 'A', 'S', 'O', '!', '->', '!', '!', '\n']
asosystest.py: Failed tests: [2]
```

Cómo podemos ver, la salida de `asosystest` nos indica con `-->` cuál es el primer carácter en el que la salida esperada y la salida producida discrepan. En este caso, vemos que en lugar de un `\n`, que era lo esperado, se ha recibido un `!!\n`.

La utilidad `asosystest` también verifica que el código de retorno es correcto. Por ejemplo, si sustituimos `exit(EXIT_SUCCESS)` por `exit(EXIT_FAILURE)` en `tarea.c`:

```
$ ./asosystest.py -i tarea.json -t 2 -d
asosystest.py: Version: v0.23.1
asosystest.py: Created temporary directory: '/tmp/tmpivuwgukx'.
asosystest.py: Successfully copied binaries: '['tarea']'.
asosystest.py: Successful os.chdir('/tmp/tmpivuwgukx').
asosystest.py: Successfully executed setup commands '['touch fl']'.

asosystest.py: Archivo de configuración para validación de 'tarea'.T02:
Result      : ERROR!
asosystest.py: Archivo de configuración para validación de 'tarea'.T02:
Command     : './tarea # Argumentos'
asosystest.py: Archivo de configuración para validación de 'tarea'.T02:
asosystest.py: Archivo de configuración para validación de 'tarea'.T02:
Expected (rc 0): ['H', 'e', 'l', 'l', 'o', ',', ' ', ' ', 'A', 'S', 'O', '!',
'\n', '-->']
asosystest.py: Archivo de configuración para validación de 'tarea'.T02:
asosystest.py: Archivo de configuración para validación de 'tarea'.T02:
Produced (rc 1): ['H', 'e', 'l', 'l', 'o', ',', ' ', ' ', 'A', 'S', 'O', '!',
'\n', '-->']
asosystest.py: Archivo de configuración para validación de 'tarea'.T02:
Return code mismatch [0 != 1]!
asosystest.py: Failed tests: [2]
```

Las utilidades `genera_bytes.py` y `genera_lineas.py`

La utilidad `genera_bytes.py` produce un flujo de bytes aleatorios, mientras que la utilidad `genera_lineas.py` crea una secuencia de líneas de texto aleatorias de tamaño arbitrario. Estas utilidades se emplean para generar la entrada necesaria para validar las tareas.

La biblioteca `libasosysintercept.so.0.1`

El archivo `asosysintercept.c` contiene el código fuente de la biblioteca compartida `libasosysintercept.so.0.1`. Esta biblioteca permite capturar llamadas al sistema y funciones de biblioteca para alterar su comportamiento:

- Inyectando errores, por ejemplo, haciendo que una llamada a `write()` devuelva `-1` y actualice el valor de `errno`.
- Forzando la realización de lecturas o escrituras parciales cuando se llama a `read()` y `write()`, respectivamente.
- Deshabilitando ciertas funciones de biblioteca o llamadas al sistema como, por ejemplo, `strcat`.

Nota: La lista completa de funciones de biblioteca y llamadas al sistema deshabilitadas se puede encontrar en la sección `NOOP` del archivo fuente `asosysintercept.c`.

Para compilar la biblioteca, hay que usar la utilidad `make`:

```
$ make libasosysintercept.so.0.1
cc -ggdb3 -Wall -Werror -Wno-unused -std=c11 -c -o asosysintercept.o
asosysintercept.c
cc --shared -fPIC -Wl,--no-as-needed -ldl asosysintercept.o -o
libasosysintercept.so.0.1
```

Cómo se puede comprobar, la compilación de la biblioteca requiere opciones distintas a las usadas para compilar los ejemplos, `minigrep.c` o `watchdir.c`. Por tanto, la compilación de `asosysintercept.c` fallará con la configuración de compilación de `.vscode/launch.json` de la copia de Visual Studio Code disponible en el Aula Virtual:

```
Iniciando la compilación...
/usr/bin/gcc -ggdb3 -Wall -Werror -Wno-unused -std=c11
/path/to/asosysintercept.c -o /path/to/asosysintercept
/usr/bin/ld: /usr/lib/gcc/x86_64-linux-gnu/11/../../../../x86_64-linux-
gnu/Scrt1.o: en la función `_start':
(.text+0x1b): referencia a `main' sin definir
collect2: error: ld returned 1 exit status

La compilación ha finalizado con errores.

* El proceso del terminal finalizó con el código de salida -1.
* Las tareas reutilizarán el terminal, presione cualquier tecla para
cerrarlo.
```

Para usar la biblioteca, basta con anteponer al binario `LD_PRELOAD=./libasosysintercept.so.0.1`, tal y como puede verse en los archivos de configuración `minigrep.json` y `watchdir.json`. Para más detalles, véase [Linux man pages online](#) • [System Manager's Manual](#) • [ld.so](#).

Requerimientos

Las utilidades `asosystest.py`, `genera_bytes.py` y `genera_lineas.py` descritas en el presente documento requieren **Python $\geq$ 3.10**. Nótese que en los laboratorios, Ubuntu 22.04 LTS incluye Python 3.10.12.